Explicit circuits for the Walsh–Hadamard transform below the textbook operation count

Jacob Sussman

11 October 2026

Not peer reviewed; not submitted anywhere.

Abstract

The textbook program for the Walsh–Hadamard transform of $N = 2^k$ numbers uses $N \log_2 N$ additions and subtractions. I give explicit circuits that use fewer, in two ways of counting. Count (G): one operation is any $au + bv$, and multiplying by a constant is free. Here the 8-point transform takes 19 operations (textbook 24; the smallest count I found in print is 22), and a recursive family takes $\frac{3}{4}N \log_2 N + \frac{1}{4}N$ for every $N \geq 4$: 52 at 16 points, 304 at 64. The smallest constant in front of $N \log_2 N$ that I found in print is 7/8 (Coppersmith, Feig and Linzer, 1994); every line of my circuits is one operation in the count of that paper, so the comparison is like for like. Count (O): every addition, subtraction and multiplication by a constant costs one. Here the exact transform on $N = 8^j$ points takes at most $\frac{5}{6}N \log_2 N + N$ operations: 4,352 against 4,608 at 512 points, the first size where it is below the textbook (28 against 24 at 8 points). The smallest constant that I found in print is 23/24 (Alman and Rao, 2023). The count-(O) statement is checked in the proof assistant Lean. The others have short proofs, given here in full; AI coding agents wrote them, and no person has checked them line by line. The circuits of all these statements were replayed exactly by program up to 16,384 points. This is a better constant, not a better exponent: algorithms with $O(N(\log N)^z)$ operations, $z < 1$, have been announced with proofs checked by Lean, and they are smaller for large N. The circuits here are explicit and are small at sizes one can write down. They were found by computer searches run with the same agents. "New" means that I did not find them in the literature I searched. Not everything that bears on this was read: Section 7 says what was read in full, what in part and what from abstracts only. A timing test on one machine found no speed-up; hardware figures are in Remark 10.

1 Introduction

The transform and the textbook count. Let $N = 2^k$. Number the inputs $x_0, \ldots, x_{N-1}$. The *Walsh–Hadamard matrix* H_N has, in row i and column c (both numbered from 0), the entry $+1$ if i and c, written in binary, have an even number of 1-bits in common, and -1 otherwise. The *Walsh–Hadamard transform* of x is the list $H_N x$; I say *exact* when I mean $H_N x$ itself, with no factor in front. Since $H_{2N} = \left(\begin{smallmatrix} H_N & H_N \\ H_N & -H_N \end{smallmatrix}\right)$, the transform can be computed in k layers, each of $N/2$ additions and $N/2$ subtractions. This is the textbook program, the fast Walsh–Hadamard transform. It uses $N \log_2 N$ operations. When a count is $\gamma N \log_2 N$ plus a multiple of N, I call γ its *rate*; the textbook has rate 1.

Two ways of counting. A *circuit* is a list of lines. Each line computes one new value from inputs and from values of earlier lines, and some of the values are named as the outputs. Every number in this paper belongs to one of two counts.

- **Count (G).** A line is $w = au + bv$ with any constants a, b, and costs one. Multiplying by a constant alone is free. This is the natural count for hardware in which a multiplication comes together with the addition in one instruction ("multiply/add"), and for linear circuits whose gates have two inputs, where it is half the number of wires (a *wire* is one input of one gate).
- **Count (O).** A line is $u + v$, $u - v$ or $c\,u$ with a constant c, and each costs one. This is the number of arithmetic operations. It is the count of Alman and Rao [3], and the gate set of the circuit model in OpenAI's Lean development [10].

The textbook costs $N \log_2 N$ in both counts. A circuit of count (O) is also a circuit of count (G) of the same size, and one line of count (G) can be written with at most three lines of count (O). No count-(O) circuit of this paper contains a bare change of sign, so it does not matter whether one charges for that.

The numbers. The inputs and the constants are rational, real or complex numbers. Proposition 1 (its left side) and Theorem B use only the constants 1, -1, 2 and -2. Theorem A and Corollary 2 also use $\frac{1}{2}$. Proposition 6 is stated for these three kinds of numbers; its proof uses of them only that they form a field in which $2 \neq 0$.

What is in print. *Count* (G). Coppersmith, Feig and Linzer [4] say on page 969 what they count: a multiply/add, which "computes $\pm a \pm bc$ where a, b and c are real numbers", costs one, and so does a plain addition or a plain multiplication. They give $\frac{7}{8}\, n \log_2 n$ operations for the Hadamard transform of length n when n is a power of four, and $n/8$ more when n is an odd power of two. By these formulas that is 7 operations at 4 points, 22 at 8 and 56 at 16. They print a 4-point circuit and the two formulas. They give no circuit of their own for 8 or for 16 points; for 16 points they say that eight 4-point transforms can be used, with the same number of operations (page 970). They claim no lower bound.

Count (G) allows a constant on both operands, which is more than a multiply/add. The circuits of this paper do not use that: every line has two terms and at most one multiplication by a constant (Remark 7). So each line is exactly one operation of [4], and the comparisons with that paper are like for like, in its own count: 19 against 22 at 8 points, 52 against 56 at 16, and the rate 3/4 against 7/8.

Seven operations at 4 points are due to Coppersmith, Feig and Linzer [4, page 969]. They compute one value that differs from three of the four outputs in a single sign, and get each of those outputs by adding twice one input. A 4-point circuit with 7 operations was found again several times: by genetic programming (Samadi, Suzukake and Iwakura [13], known from its abstract), in a patent of Reshef and Alrod [12], and by Alman and Rao [3], who use 7 additions or subtractions and one halving. Feig wrote, in a chapter dated 1999, that the 4-point result "has not been extended, and it may in fact be rather hard to extend except in most trivial (and uninteresting) ways" [5]. The smallest count at 8 points that I found in print is 22, and it appears twice. The formula of [4] for odd powers of two gives 22 at 8 points: one textbook layer of 8 operations and two 4-point circuits of 7 (they print the formula, not the number). And the 8-point step of Alman and Rao uses 22 additions or subtractions and one halving. The halving is free in count (G), so their 23 operations of count (O) are 22 of count (G).

Count (O). Alman and Rao [3, Theorem 1.1] prove that for any field and any N that is a power of two, $H_N x$ can be computed with at most $\frac{23}{24} N \log_2 N + \frac{13}{12} N$ field operations. They write that, to their knowledge, this is the first improvement on the operation count $N \log N$ of the textbook program. The proof of their Theorem 6.2 gives the exact count $\frac{23}{24} N \log_2 N + \frac{N}{24}(\log_2 N \bmod 3) + N - 1$. Their method writes H_8 as the sum of a matrix of low rank (one whose rows are all combinations of a few rows) and a sparse matrix (one with few non-zero entries), and halves once; the resulting step costs 23 operations per 8 points. These statements are in version 2 of their arXiv text. The idea of a single halving, used below, is theirs.

Other measures. Alman [2] and later papers count the wires of circuits that are built in layers and whose gates may have many inputs; a value that passes a layer unchanged costs a wire there. His abstract gives a bound for every fixed number of layers and, with no limit on their number, $(1.81 + o(1))\, N \log_2 N$ wires. That is a third count, which I do not treat, and the numbers of this paper do not convert into it line by line; Alman and Rao write that the algorithms of [2] "do not give improved arithmetic operation counts" [3].

The count of Grigoryan and Agaian. Grigoryan and Agaian [7] state that their algorithm computes the Hadamard transform on $N = 2^r$ points with $6 \cdot 2^r - r^2 - 3r - 6$ additions and subtractions (page 181), "no more than six additions per sample" (page 179). That count is too small. As the program printed on page 181 carries it out, the algorithm applies a "paired transform" of length N, which costs $2N - 2$ additions and subtractions, and this splits the transform into transforms of lengths $N/2, N/4, \ldots, 2, 1$, which are computed the same way. So the number $T(N)$ of its operations obeys

$$T(N) = (2N - 2) + T(N/2) + T(N/4) + \cdots + T(1), \qquad T(1) = 0,$$

and the solution is $T(N) = N \log_2 N$ exactly (by induction, because $\sum_{i<r} i\, 2^i = (r-2)\, 2^r + 2$). The paper does not print this recursion; it is the count of the algorithm, written down from that page. The stated count agrees with it at 2, 4 and 8 points (2, 8, 24) and is smaller from 16 points on (62 against 64). From 4,096 points on it is also below $\frac{1}{2} N \log_2 N$ (24,390 against 24,576). That number is a lower bound for circuits that use only additions and subtractions, by the determinant argument of Morgenstern [9], taken here as Alman and Rao quote it [3] (the determinant of H_N is $\pm N^{N/2}$); and the introduction of [7] itself quotes $0.5\, N \log N$ as a lower bound for the Hadamard transform (page 172). So the algorithm has the textbook count in another order, and the count stated in [7] is not a count below $N \log_2 N$. The program `check.py` of Section 5 prints the stated count beside the two other numbers for every size up to 16,384 points. A program that the agents wrote earlier from a later description of the algorithm by the first author [8] also used exactly $N \log_2 N$ additions and subtractions at every size from 2 to 4,096 points; that program is not in the repository. Frigo and Johnson [6] published a comment on the part of [7] that concerns the Fourier transform. Only its abstract was read, and it does not speak of the count for the Hadamard transform. Nothing is said here about the counts of [7] for the Fourier transform.

The exponent. Two preprints of OpenAI of 25 September 2026 [10, 11] announce, with a Lean development, that the exact discrete Fourier transform of length n can be computed in $O(n(\log n)^z)$ operations with a fixed $z < 1$. A proof, checked by Lean, for the Walsh–Hadamard and Fourier transforms with $1 - z \approx 8.7625 \times 10^{-4}$ is in an earlier public repository of mine [18], which this paper does not use. Both count the steps of a machine with exact arithmetic, not the gates of a circuit, so I compare orders of growth only. For large N these are smaller than $c\,N \log N$ for every constant $c > 0$, and so smaller than every count in this paper.

The results. Table 1 lists them. In this paper "new" always means: I did not find it in the literature I searched. Section 7 says what was searched, and what bears on the question and was not read.

statement	count, sizes	this paper	found in print	how it is checked
Prop. 1	(G), $N = 8$	19	22	replay; Table 2
Thm. B	(G), every $N = 2^k \geq 4$	$\frac{3}{4} N \log_2 N + \frac{1}{4} N$	$\leq \frac{7}{8} N \log_2 N + \frac{1}{8} N$ [4]	proof; replay
Thm. A	(O), $N = 8^j$	$\frac{5}{6} N \log_2 N + N$	$\frac{23}{24} N \log_2 N + N - 1$ [3]	proof; Lean; replay
Cor. 2	(O), every $N = 2^k \geq 8$	$\leq \frac{5}{6} N \log_2 N + \frac{4}{3} N$	$\leq \frac{23}{24} N \log_2 N + \frac{13}{12} N$ [3]	proof; replay
Prop. 6	(G) and (O), $N \geq 4$	at least $2N - 1$	not searched	proof

Table 1: The results, all for the exact transform $H_N x$. "Found in print" is the smallest count that I found in the literature I searched; for Theorem B it is $\frac{7}{8} N \log_2 N$ when k is even and $\frac{1}{8} N$ more when k is odd. "Replay" means: a program generated the circuit and ran it in exact arithmetic on every *unit input* (one input 1, the others 0), or made one run that is equivalent to this, for every size up to $N = 2^{14}$. Proposition 1 is short enough to check by hand from Table 2. Section 5 says who wrote and who read the proofs.

In count (O) the circuits are below the textbook from 512 points on. In count (G) they are below it from 4 points on, and below the counts I found in print from 8 points on.

The circuits as text files, the programs that check them, the Lean files and the source of this paper are in one repository [17]. "The repository" always means that one.

2 The eight-point circuit

Table 2 gives the circuit. The middle columns show each value as its eight coefficients in $x_0, \ldots, x_7$.

line, count (G)	x_0	x_1	x_2	x_3	x_4	x_5	x_6	x_7	centre halved, count (O)
$g_1 = x_0 + x_7$	+	·	·	·	·	·	·	+	$g_1 = x_0 + x_7$
$g_2 = x_1 + x_6$	·	+	·	·	·	·	+	·	$g_2 = x_1 + x_6$
$g_3 = x_2 - x_5$	·	·	+	·	·	−	·	·	$g_3 = x_2 - x_5$
$g_4 = x_3 - x_4$	·	·	·	+	−	·	·	·	$g_4 = x_3 - x_4$
$g_5 = g_1 - g_2$	+	−	·	·	·	·	−	+	$g_5 = g_1 - g_2$
$g_6 = g_3 + g_4$	·	·	+	+	−	−	·	·	$g_6 = g_3 + g_4$
$g_7 = g_5 - g_6$	+	−	−	−	+	+	−	+	$g_7 = g_5 - g_6$, then $h = \frac{1}{2} g_7$
$g_8 = g_7 - 2x_7$	+	−	−	−	+	+	−	−	$h_8 = h - x_7$
$g_9 = g_7 - 2x_5$	+	−	−	−	+	−	−	+	$h_9 = h - x_5$
$g_{10} = g_8 + 2x_6$	+	−	−	−	+	+	+	−	$h_{10} = h_8 + x_6$
$g_{11} = g_9 - 2x_4$	+	−	−	−	−	−	−	+	$h_{11} = h_9 - x_4$
$y_7 = g_{10} + 2g_4$	+	−	−	+	−	+	+	−	$z_7 = h_{10} + g_4$
$y_1 = g_{10} + 2g_3$	+	−	+	−	+	−	+	−	$z_1 = h_{10} + g_3$
$y_6 = g_{11} + 2g_2$	+	+	−	−	−	−	+	+	$z_6 = h_{11} + g_2$
$y_0 = -g_{11} + 2g_1$	+	+	+	+	+	+	+	+	$z_0 = g_1 - h_{11}$
$y_2 = g_8 + 2x_1$	+	+	−	−	+	+	−	−	$z_2 = h_8 + x_1$
$y_3 = g_9 + 2x_3$	+	−	−	+	+	−	−	+	$z_3 = h_9 + x_3$
$y_4 = y_2 + 2g_6$	+	+	+	+	−	−	−	−	$z_4 = z_2 + g_6$
$y_5 = -y_3 + 2g_5$	+	−	+	−	−	+	−	+	$z_5 = g_5 - z_3$

Table 2: The 8-point transform in 19 operations of count (G) (left), and $\frac{1}{2} H_8 x$ in 20 operations of count (O) (right). Middle: the coefficients of the value on the left; + is +1, − is −1, · is 0. On the right the last twelve values are half of those on the left: $h_i = \frac{1}{2} g_i$ and $z_i = \frac{1}{2} y_i$.

Proposition 1. *The* 19 *lines on the left of Table 2 compute the exact transform: y_i is row i of H_8 applied to x, for $i = 0, \dots, 7$. The* 20 *lines on the right (*19 *additions or subtractions and one halving) compute $z_i = \frac{1}{2} y_i$.*

Proof. Check the table row by row. A line has two terms: its first operand, with a minus sign in two lines, and its second operand times the constant shown. The pattern of the line is the sum of the patterns of the two terms. In each of the last twelve lines the second term is non-zero only in columns where the first term has the opposite sign, and there it is ± 2; so the line is the first term with exactly those signs flipped. The eight rows named y_i are the rows of H_8: for example y_5 has -1 exactly in the columns $1, 3, 4, 6$, the numbers that share an odd number of 1-bits with $5 = 101_2$. On the right, the first seven lines are unchanged, $h = \frac{1}{2} g_7$, and each later line is the line on its left divided by 2. □

What the structure is. Seven lines merge the eight inputs into one value, g_7, which I call the *centre*. It reads all eight inputs and it is not an output. Each of the other twelve lines flips signs, by adding twice an input or twice a value that the merging left behind $(g_1, \dots, g_6)$. So every partner in those twelve lines was already paid for. In the end every output is the centre, with a sign, plus twice a small sum:

$$\begin{aligned} y_2 &= g_7 + 2(x_1 - x_7), & y_0 &= g_7 + 2(x_1 + x_2 + x_3 + x_6), \\ y_3 &= g_7 + 2(x_3 - x_5), & y_1 &= g_7 + 2(x_2 - x_5 + x_6 - x_7), \\ y_4 &= -g_7 + 2(x_0 - x_6), & y_6 &= g_7 + 2(x_1 - x_4 - x_5 + x_6), \\ y_5 &= g_7 + 2(x_2 - x_4), & y_7 &= g_7 + 2(x_3 - x_4 + x_6 - x_7). \end{aligned}$$

Two of the 19 lines, y_0 and y_5, have a minus sign on their first operand. All constants are ± 1 and ± 2, and all 19 values have coefficients in $\{0, +1, -1\}$.

In count (O) the 19 lines as they stand would cost 31: 19 additions or subtractions and 12 doublings. Halving the centre once, right after the line g_7, removes every doubling. This single halving is the device of Alman and Rao [3]; their 8-point step costs 23 operations where this one costs 20.

3 Count (O): the exact transform in $\frac{5}{6} N \log_2 N + N$ operations

Theorem A. *Let $j \geq 1$ and $N = 8^j$. There is a circuit with at most*

$$20\, j\, 8^{j-1} + 8^j \;=\; \tfrac{5}{6}\, N \log_2 N + N$$

operations of count (O) *that computes the exact transform $H_N x$.*

Proof. Write the number of a point with $3j$ binary digits, and cut them into j groups of three. Row r, column c of H_N is $(-1)^{\text{number of common 1-bits}}$, and the common 1-bits are counted group by group. So H_N is the product of j matrices $L_1, \dots, L_j$ that commute, where L_i acts as H_8 on the i-th group of digits and leaves the other digits alone: L_i splits the N points into $N/8$ sets of eight points that differ only in the i-th group, and applies H_8 to each set. (In matrix terms, $H_N = H_8 \otimes \cdots \otimes H_8$ and $L_i = I \otimes \cdots \otimes H_8 \otimes \cdots \otimes I$.)

Now apply the 20-operation circuit of Proposition 1 in place of H_8: for $i = 1, \dots, j$ in turn, run it on each of the $N/8$ sets of the i-th group, with the current values as inputs. This is j layers of $20 \cdot N/8$ operations, and by Proposition 1 it computes $\frac{1}{2} L_j \cdots \frac{1}{2} L_1 x = 2^{-j} H_N x$. Multiply each of the N values by 2^j. The total is $20\, j\, N/8 + N = 20\, j\, 8^{j-1} + 8^j$ operations, and $20\, j\, N/8 = \frac{20}{24} N \cdot 3j = \frac{5}{6} N \log_2 N$. □

Corollary 2. *Let $k = 3m + r$ with $m \geq 1$ and $r \in \{0, 1, 2\}$, and $N = 2^k$. Then $H_N x$ can be computed exactly with $N(\frac{5}{2} m + r + 1)$ operations of count* (O)*. This is $\frac{5}{6} N \log_2 N + N$ for $r = 0$, $\frac{5}{6} N \log_2 N + \frac{7}{6} N$ for $r = 1$ and $\frac{5}{6} N \log_2 N + \frac{4}{3} N$ for $r = 2$. It is below the textbook's $N \log_2 N$ exactly when $m \geq 3$, that is, from* 512 *points on.*

Proof. As in the proof of Theorem A, use m layers of the 20-operation circuit on $3m$ of the digits ($\frac{20}{8} mN$ operations), one textbook layer for each of the r other digits (rN operations), and N multiplications by 2^m. Since $m = (k - r)/3$, the total is $\frac{5}{6} Nk + (1 + \frac{r}{6})N$. It is smaller than $Nk = N(3m + r)$ exactly when $\frac{5}{2} m + 1 < 3m$. □

N	textbook $N\log_2 N$	Alman and Rao, exact count	this paper (Cor. 2)	ratio to textbook
8	24	30	28	1.167
16	64	77	72	1.125
32	160	187	176	1.100
64	384	431	384	1.000
128	896	991	896	1.000
256	2,048	2,239	2,048	1.000
512	4,608	4,927	**4,352**	0.944
1,024	10,240	10,879	**9,728**	0.950
2,048	22,528	23,807	**21,504**	0.955
4,096	49,152	51,199	**45,056**	0.917

Table 3: Count (O), exact $H_N x$. "Alman and Rao, exact count" is $\frac{23}{24}N\log_2 N+\frac{N}{24}(\log_2 N \bmod 3)+N-1$, from the proof of their Theorem 6.2. Bold: below the textbook.

The comparison. Table 3 has the numbers from 8 to 4,096 points. Three things can be read from the formulas.

- The count of Corollary 2 is below the textbook at every $N \geq 512$, equal to it at 64, 128 and 256 points, and above it at 8, 16 and 32 points. Below 512 points the textbook is the smallest exact count in count (O) that I know.
- It is below Alman and Rao's exact count at every $N \geq 8$: the difference is $\frac{3}{8}mN - 1$.
- Alman and Rao's exact count is above the textbook at every size in the table. By its formula it is first below the textbook at $N = 2^{24}$, by one operation; their stated bound $\frac{23}{24}N\log_2 N + \frac{13}{12}N$ is first below it at $N = 2^{27}$. Their theorem is stated for any field. Their circuit and mine both divide by 2, so both need 2 to be different from 0 (where $2 = 0$ every entry of H_N is 1 and the transform is the sum of the inputs). The Lean statement is over the complex numbers.

Remark 3 (if a common factor is allowed). Without the last N multiplications the same circuits compute $2^{-m}H_N x$ in $N(\frac{5}{2}m + r)$ operations of count (O): 20 at 8 points, 56 at 16, 320 at 64, 3,840 at 512. This is below $N\log_2 N$ from 8 points on. For even k the family of Section 4, with its pieces kept at suitable powers of $\frac{1}{2}$, gives $H_N x/\sqrt{N}$ in $\frac{7}{8}N\log_2 N + \frac{1}{4}N$ operations of count (O), 60 at 16 points; this was replayed by the programs of the repository [17] up to 16,384 points, and I do not prove it here.

That rate, 7/8, is above 5/6. On 11 October 2026 the agents of Section 7 looked for a way to bring the rate 3/4 of the family into count (O), and found none. First, the lines of the family were kept and each value was held multiplied by a power of 2 that could be chosen freely; a value that is needed with two different powers costs one more operation. The least count of this kind, with the exponents taken from a fixed range, was computed exactly at 4 to 64 points. It is the 60 above at 16 points and 352 at 64, against 56 and 320 for the layers of this section, and an argument puts it at $\frac{7}{8}N\log_2 N + \frac{1}{8}N$ or more at every size. Second, six ways of putting levels of the family together with the 8-point circuit were built and replayed, up to 4,096 points; none costs less than the layers. These are computations and an argument of the agents, not in Lean and not checked by a person, and they say nothing about circuits that are wired in another way. In particular, no direct search has been made for a 16-point circuit that computes a multiple of $H_{16}x$ with 53 or fewer operations of count (O); used in layers, such a circuit would have a rate below 5/6. The best I know has 56.

4 Count (G): a family with $\frac{3}{4}N\log_2 N + \frac{1}{4}N$ operations

Theorem B. *For every $N = 2^k$ with $k \geq 2$ there is a circuit with exactly*

$$\tfrac{3}{4}\,N\log_2 N + \tfrac{1}{4}\,N$$

operations of count (G) *that computes the exact transform $H_N x$. All its constants are ± 1 and ± 2.*

The counts are 7, 20, 52, 128 and 304 for $N = 4, \ldots, 64$; Table 4 has more. At 4 points the circuit has the 7 operations of Coppersmith, Feig and Linzer [4], in another wiring of the same idea: theirs has 4 additions or subtractions and 3 lines with a doubling, and the family at 4 points has 3 and 4. At 8 points the family has 20 operations, one more than the circuit of Section 2; from 16 points on it is the smallest count (G) that I know.

Words. A *block* of size $m = 2^i$ is a run of m consecutive inputs $x_e, \dots, x_{e+m-1}$ that starts at a multiple e of m. For a block X of size m and $0 \le a < m$, the *piece* (X, a) is row a of the m-point transform applied to the block:

$$(X,\, a) \;=\; \sum_{t=0}^{m-1} (-1)^{\text{number of common 1-bits of } a \text{ and } t}\; x_{e+t}.$$

A block of size 1 has one piece, the input itself. The N outputs are the N pieces of the block of size N. A block X of size $m \ge 2$ has a lower half L and an upper half R, and for $0 \le a < m/2$

$$(X,\, a) = (L,\, a) + (R,\, a), \qquad (X,\, a + m/2) = (L,\, a) - (R,\, a). \tag{1}$$

This is the textbook step. The textbook makes all m pieces of every block, and so keeps every piece of L and every piece of R. The family keeps half of them.

Lemma 4 (the hub)**.** *Let X be a block of size $m \ge 4$ with halves L, R and quarters L_0, L_1, R_0, R_1, in this order. Let $0 \le d < m/4$, and write $q(Y) = (Y,\, d)$ for a quarter Y. Put*

$$A = (L,\, d), \qquad B = (R,\, d + m/4), \qquad M = A + c\,B \quad \textit{with } c \in \{+1, -1\}.$$

Let $s \in \{+1, -1\}$, and let $P_1 = (L,\, d) + s\,(R,\, d)$ and $P_2 = (L,\, d + m/4) + s\,(R,\, d + m/4)$. Then

$$P_1 = M + (s - c)\, q(R_0) + (s + c)\, q(R_1), \tag{2}$$
$$P_2 = cs\, M + (1 - cs)\, q(L_0) - (1 + cs)\, q(L_1). \tag{3}$$

So if $c = -s$ then $P_1 = M + 2s\, q(R_0)$ and $P_2 = -M + 2\, q(L_0)$, and if $c = s$ then $P_1 = M + 2s\, q(R_1)$ and $P_2 = M - 2\, q(L_1)$. By (1), P_1 and P_2 are the pieces $(X,\, d)$ and $(X,\, d + m/4)$ of X when $s = +1$, and the pieces $(X,\, d + m/2)$ and $(X,\, d + m/4 + m/2)$ when $s = -1$.

Proof. By (1) applied to L and to R, whose halves are the quarters,

$$(L,\, d) = q(L_0) + q(L_1), \quad (L,\, d + m/4) = q(L_0) - q(L_1),$$
$$(R,\, d) = q(R_0) + q(R_1), \quad (R,\, d + m/4) = q(R_0) - q(R_1).$$

Hence $M = q(L_0) + q(L_1) + c\, q(R_0) - c\, q(R_1)$, while $P_1 = q(L_0) + q(L_1) + s\, q(R_0) + s\, q(R_1)$ and $P_2 = q(L_0) - q(L_1) + s\, q(R_0) - s\, q(R_1)$. Subtracting M from P_1 gives (2). Since $c^2 = 1$, $cs\, M = cs\, q(L_0) + cs\, q(L_1) + s\, q(R_0) - s\, q(R_1)$, and subtracting it from P_2 gives (3). □

I call M a *hub*. It is not a piece: it is one piece of L on the lower half plus a different piece of R on the upper half. It is one operation away from two pieces of X, and the partners in those two operations are pieces of quarters.

The procedure $E(X, s)$. For a block X of size $m \ge 2$ and a sign s, the procedure $E(X, s)$ makes the $m/2$ pieces $(L,\, a) + s\,(R,\, a)$, $0 \le a < m/2$. By (1) these are the pieces $(X,\, a)$ with $a < m/2$ if $s = +1$, and with $a \ge m/2$ if $s = -1$.

- If $m = 2$: one line, $x_e + s\, x_{e+1}$.
- If $m \ge 4$: first run $E(L, +1)$ and $E(R, -1)$. Then, for every d with $0 \le d < m/4$, write three lines in the notation of Lemma 4: the hub $M = A + cB$ with $c = -s$ if $d < m/8$ and $c = s$ if $d \ge m/8$; then P_1; then P_2, each from M and one quarter piece as in the lemma.

Lemma 5. *For every block X of size $m = 2^i \ge 2$ and every sign s, each line of $E(X, s)$ uses only inputs and values of earlier lines, $E(X, s)$ makes the $m/2$ pieces stated, and it has exactly $E(m) = m(3i - 1)/4$ lines.*

Proof. By induction on m. For $m = 2$ the line is $(L,\, 0) + s\,(R,\, 0)$, and $E(2) = 1$. Let $m \ge 4$ and suppose the lemma holds for $m/2$ and, if $m \ge 8$, for $m/4$. Fix $d < m/4$. The piece $A = (L,\, d)$ has index $d < \frac{1}{2}(m/2)$, so $E(L, +1)$ made it. The piece $B = (R,\, d + m/4)$ has index at least $\frac{1}{2}(m/2)$, so $E(R, -1)$ made it. For the quarter pieces there are two cases. If $m = 4$, then $d = 0 < m/8$, the quarters are single inputs, and $q(L_0)$, $q(R_0)$ are inputs. If $m \ge 8$, then $E(L, +1)$ began by running $E(L_0, +1)$ and $E(L_1, -1)$, and $E(R, -1)$ began by running $E(R_0, +1)$ and $E(R_1, -1)$. By induction these made $(L_0,\, d)$ and $(R_0,\, d)$ for $d < m/8$, and $(L_1,\, d)$ and $(R_1,\, d)$ for $m/8 \le d < m/4$. So for $d < m/8$ the pieces

$q(L_0), q(R_0)$ are at hand and, with $c = -s$, Lemma 4 gives $P_1 = M + 2s\,q(R_0)$ and $P_2 = -M + 2\,q(L_0)$. For $d \geq m/8$ the pieces $q(L_1), q(R_1)$ are at hand and, with $c = s$, it gives $P_1 = M + 2s\,q(R_1)$ and $P_2 = M - 2\,q(L_1)$. Each is one line of count (G). As d runs from 0 to $m/4 - 1$, the pieces P_1, P_2 are $(L,\, a) + s\,(R,\, a)$ for $a = d$ and $a = d + m/4$, which is every $a < m/2$ once. The number of lines is $2E(m/2) + 3 \cdot \frac{m}{4} = \frac{m}{4}\big(3(i-1) - 1\big) + \frac{3m}{4} = \frac{m}{4}(3i - 1)$. □

Proof of Theorem B. Let X be the block of all N inputs, with halves L, R. Run $E(L, +1)$ and $E(R, -1)$. For every d with $0 \leq d < N/4$ write five lines, in the notation of Lemma 4 with $m = N$ and $c = +1$, so that $M = A + B$.

- If $d < N/8$: $U = M - 2\,q(R_0)$ and $V = -M + 2\,q(L_0)$; then $2A - U$ and $V + 2B$.
- If $d \geq N/8$: $U' = M + 2\,q(R_1)$ and $V' = M - 2\,q(L_1)$; then $2A - U'$ and $V' - 2B$.

The values A, B and the quarter pieces are at hand for the same reasons as in the proof of Lemma 5. In the first case Lemma 4 with $s = -1 = -c$ gives $U = (L,\, d) - (R,\, d) = (X,\, d + N/2)$ and $V = (L,\, d + N/4) - (R,\, d + N/4) = (X,\, d + N/4 + N/2)$. Then $2A - U = (L,\, d) + (R,\, d) = (X,\, d)$, and $V + 2B = (L,\, d + N/4) + (R,\, d + N/4) = (X,\, d + N/4)$, because $B = (R,\, d + N/4)$. In the second case the lemma with $s = +1 = c$ gives $U' = (X,\, d)$ and $V' = (X,\, d + N/4)$, and then $2A - U' = (X,\, d + N/2)$ and $V' - 2B = (X,\, d + N/4 + N/2)$. So the four outputs with the numbers d, $d + N/4$, $d + N/2$, $d + 3N/4$ are made, and as d runs from 0 to $N/4 - 1$ these are all N outputs. Every constant in these lines is ± 1 or ± 2. By Lemma 5 the number of lines is

$$2E(N/2) + 5 \cdot \tfrac{N}{4} = \tfrac{N}{4}\big(3(k-1) - 1\big) + \tfrac{5N}{4} = \tfrac{N}{4}(3k + 1) = \tfrac{3}{4} N \log_2 N + \tfrac{1}{4} N. \qquad \square$$

N	textbook	rate 7/8 found in print	layers of the 19	family (Theorem B)
4	8	7	7	7
8	24	22	**19**	20
16	64	56	54	**52**
32	160	144	132	**128**
64	384	336	**304**	**304**
128	896	800	736	**704**
256	2,048	1,792	1,664	**1,600**
512	4,608	4,096	3,648	**3,584**
1,024	10,240	8,960	8,320	**7,936**
2,048	22,528	19,968	18,176	**17,408**
4,096	49,152	43,008	38,912	**37,888**

Table 4: Count (G), exact $H_N x$. "Rate 7/8 found in print": the two formulas of [4], page 969, which are $\frac{7}{8} N \log_2 N$ for even k and $\frac{7}{8} N \log_2 N + \frac{1}{8} N$ for odd k (one textbook layer and two transforms of half the length). "Layers of the 19": the 19-operation circuit on three digits at a time, then a textbook layer for one digit left over or the 7-operation circuit for two. Bold: the smallest of the row, when it is from this paper.

Where the gain comes from. A level of the textbook costs N. In the family a level costs $3N/4$: for each d, one hub and two pieces in place of four pieces. The price is that a hub needs a partner, and the partner is a piece made two levels below. So each piece is used twice: once to build the hub one level up, and once, two levels up, to turn a hub into a piece. If one uses the 4-point circuit on two digits at a time, with no sharing between consecutive pairs of levels, the count obeys $T(N) = 4T(N/4) + \frac{7}{4} N$, which is the rate 7/8 of [4].

5 What has been checked, and how

The statement checked in Lean. The Lean files are in the folder `lean/` of the repository [17]; those written for this result are in `lean/Work/SmallWHT/`. The following statement was checked by Lean 4 (version 4.34.1) with the library Mathlib:

```
theorem OAI.ExactFourier.WHT8Half.hadamard_pow8_circuit (j : ℕ) :
    ∃ C : Circuit (2 ^ (3 * j)),
      C.Computes (hadamard (3 * j)) ∧ C.size ≤ 20 * (j * 8 ^ (j - 1)) + 8 ^ j
```

Here `Circuit n`, `size` and `Computes` are OpenAI's definitions [10]: a circuit on n inputs is a list of gates `add i j`, `sub i j` and `scale c i` with c any complex number, together with a choice of n outputs; `size` is the number of gates; `C.Computes A` says that for every complex input x the outputs are Ax. The model also offers the constant 0 as a value that costs nothing; this changes no count here. The subtraction `j - 1` is that of natural numbers (so $0 - 1$ is 0). The prefix `OAI` in the name is the name space of OpenAI's Lean files, which my files extend; the files of this paper are mine, and this is not a project of OpenAI. The matrix `hadamard k` is defined in my files by

```
def hadamard (k :  ℕ) :  Matrix (Fin (2 ^ k)) (Fin (2 ^ k)) ℂ :=
    fun j l => ∏ i ∈ Finset.range k,
        if j.val.testBit i && l.val.testBit i then (-1 :  ℂ) else 1
```

which is H_{2^k} as defined in the Introduction. So the Lean statement is Theorem A. A second theorem in the same files states the bound as $\frac{5}{6}N \log_2 N + N$ with real numbers.

What this covers. In the Lean proof the 20 operations on the right of Table 2, the same operations in another order, are data, and Lean checks that they compute $\frac{1}{2}H_8$. The step to 8^j points uses OpenAI's Lean development of circuit costs for tensor products; the 44 files of theirs that the proof needs are in the repository, unchanged. The folder `lean/` of the repository was built once, on one Linux machine, in a new folder with a new installation of Lean and with Mathlib's compiled files taken from its cache; the file `VERIFY.md` of the repository gives the commands. Lean reported no error, and the theorem depends only on the three standard axioms `propext`, `Classical.choice` and `Quot.sound`. A separate file holds the definitions and the statement with no proof. A comparison script in the repository, written by the agents, checked that the theorem that is proved is exactly that statement, with the same definitions. The comparator, which is Lean's official program for this comparison, accepted the proof as a proof of that statement in an earlier run, in the working copy in which the files were written; it was not run on the repository itself. The comparator is meant to run inside a *sandbox*, a program that shuts the checked files off from the rest of the machine; that run used a simple stand-in for it. Two controls were run on copies of files of the repository: with one gate of the 8-point circuit changed, Lean refused the proof, and with the bound 20 changed to 19, the comparison script refused the statement, as the comparator had done in its run.

What it does not cover. Nothing in count (G) is in Lean: not the 19, not Theorem B, not Proposition 6. Sizes that are not powers of 8 (Corollary 2) are not in Lean. Above 8 points the Lean proof builds the circuit inside the logic and writes out no list of gates. The definition of `hadamard` was written for this statement. A short Lean proof links it to a function `wht` that writes each output as a sum of the inputs with signs; both use the same formula for the signs, so this is a check of the wording and not a second definition. No person has yet read the Lean files, the statement included.

Exact replay by program. Two sets of Python programs check the circuits. Both work in exact arithmetic and run every circuit on all N unit inputs, or make one run that is equivalent to this, which determines a linear map.

- The programs of the repository [17], in the folder `tools/circuits/`, write every circuit as a text file, read it back and check it against the definition of H_N: the circuits of Table 2, the family, and the layered circuits of Section 3, for every size up to 4,096 points on one machine and up to 16,384 on another. One command, run in the top folder of the repository, repeats the run up to 4,096 points: `python3 -B tools/circuits/run.py`. The folder `tools/circuits/fast/` has the circuits as working programs in Python and C, with tests. The repository is under the Apache 2.0 licence.
- A second program, `check.py`, in the folder `paper/` of the repository, was written from the text of this paper. For Theorem B it follows the proof line by line and compares every line, when it is made, with the piece that the proof says it is; it also confirms that the values are the same as those of the first generator. It was run for every k from 2 to 14. It also replays the layers of Corollary 2 for $k = 3, \ldots, 14$, recomputes every number in the tables, tests that every line of the 19 and of the family is one operation of [4] (Remark 7), and prints the numbers on the count of [7] that the Introduction quotes. It needs the Python library numpy. In the top folder of the repository the command is `python3 -B paper/check.py 12 tools/circuits`; the number is the largest k.

A third program, `lowerbound_check.c`, in the same folder `paper/`, is about the proof of Proposition 6. Step 3 of that proof shows that the first $N - 1$ lines of a circuit with $2N - 2$ lines form a tree whose leaves are the inputs. The program goes through all such trees with 4 and with 8 leaves (15 and 135,135

trees, each with every row at the root) and confirms Steps 4 and 5 at those sizes: no case reaches more than one further row.

A replay is a check by a program, not a proof. For sizes above 16,384 points the statements rest on the proofs in this paper, which use nothing from outside it; Proposition 1 can be checked by hand from Table 2.

The proofs. The proofs in this paper were written by the AI agents of Section 7. The proof of Theorem B was read by a second agent, and `check.py` follows it line by line up to 16,384 points. Proposition 6 and its proof were written on the morning of 11 October 2026, after the literature search; a second agent read the proof, and the only check of it by program is that of `lowerbound_check.c`, for Steps 4 and 5 at 4 and 8 points.

6 Remarks

A lower bound, and the best counts I know at small sizes

Proposition 6. *Let $N = 2^k \geq 4$. Every circuit of count* (G)*, with rational, real or complex constants, that has for each row of H_N a value equal to a non-zero multiple of that row applied to x, has at least $2N - 1$ operations. The same holds in count* (O)*, whose lines are special lines of count* (G)*.*

Proof. Let the lines be $g_1, \ldots, g_S$, in order. The *support* of a value is the set of inputs with a non-zero coefficient in it. A line *depends on* its two operands and on everything they depend on; this is a matter of the wiring only, so a constant equal to 0 does not remove an operand. Call the *ancestors* of a line the lines that it depends on, itself included, and its *cone* the set of inputs that it depends on. The support is contained in the cone.

Step 1. A line with t ancestors has a cone of at most $t + 1$ inputs. Indeed, take as vertices the t ancestors and the inputs of the cone, and draw an edge from each operand of each ancestor to that ancestor. This graph is connected, because every vertex leads to the line. It has at most $2t$ edges, and a connected graph has at least one edge fewer than it has vertices. If the cone has exactly $t + 1$ inputs, there are exactly $2t$ edges and the graph is a tree.

Step 2. Every output has all N inputs in its support, so it is a line, and by Step 1 a line with full support is g_i with $i \geq N - 1$. Different rows of H_N are not multiples of each other, so the N outputs are N different lines, all among $g_{N-1}, \ldots, g_S$. Hence $S \geq 2N - 2$.

Step 3. Suppose $S = 2N - 2$. Then the outputs are exactly $g_{N-1}, \ldots, g_{2N-2}$; the line g_{N-1} has full support, all of $g_1, \ldots, g_{N-1}$ are its ancestors, and the graph of Step 1 is a tree with root g_{N-1}. Call the inputs and $g_1, \ldots, g_{N-2}$ the *tree values*; the cone of an input is the input itself. In a tree the two operands of a line have disjoint cones, and the cone of the line is their union. Let g_{N-1} be λ times row a, written $\lambda\rho_a$. Going down from the root one sees that every tree value w is a non-zero multiple of ρ_a restricted to the cone of w (all other coefficients set to 0): if a line $\alpha u + \beta v$ is such a multiple, then on the cone of u it equals αu, because v is zero there. The cones of the tree values are *nested*: any two are disjoint, or one contains the other. Each of them is neither empty nor the set of all inputs.

Step 4. Take an output $g_i = \alpha u + \beta v$ with $i \geq N$; it is $\lambda'\rho_b$ with $b \neq a$. Each of u, v is an earlier output or a tree value. They are not both outputs, because the rows of H_N are linearly independent. Two facts are used: two different rows of H_N agree in exactly $N/2$ columns, and no row is minus another, since all rows begin with $+1$. If u is an output, a multiple of ρ_e with $e \neq b$, and v is a tree value with cone C (or the other way round), then outside C the row ρ_b is a multiple of ρ_e, so $\rho_b = \varepsilon\rho_e$ there with one sign ε. On C the difference $\rho_b - \varepsilon\rho_e$ is a multiple of ρ_a. It is not the zero multiple, since $\rho_b \neq \varepsilon\rho_e$; so it is nowhere 0 on C. Hence C is exactly the set of the $N/2$ columns where ρ_b and $\varepsilon\rho_e$ differ, and on C the difference is $2\rho_b$, so ρ_b is a multiple of ρ_a on C. If u and v are both tree values, their cones cover all inputs, because g_i has full support; so, being nested, they are complementary sets C and C'; then $\rho_b = \pm\rho_a$ on C and $\rho_b = \mp\rho_a$ on C', and both sets have $N/2$ elements. In both cases some tree value has a cone C of size $N/2$ on which the product $\rho_a\rho_b$, taken column by column, is constant. This product is a row of H_N other than row 0, so it has $N/2$ entries of each sign, and it is therefore constant on C and on the complement of C, with opposite signs.

Step 5. Among nested sets at most two have size $N/2$, and if there are two they are complementary. A set and its complement give the same product row, hence the same b. So at most one output besides g_{N-1} can be made. But there are $N - 1 \geq 3$ of them. So $S \geq 2N - 1$. □

The bound $2N-2$ is the obvious one: $N-1$ lines are needed before any value can contain all inputs, and each further output needs a line. The proposition adds one. The bound is elementary, and no search of the literature was made for it, so I do not say that it is new. The paper [4] claims no lower bound and does not say whether 7 is the least at 4 points.

For $N=4$ the bound is 7, so by Proposition 6 no circuit of count (G) for 4 points has fewer operations than the 7-operation circuits. For $N=8$ and $N=16$ the bound is 15 and 31, and the best circuits I know have 19 (Section 2) and 52 (Theorem B) operations. The exact minimum at 8 points is open: it is between 15 and 19. The searches described in Section 7 found no circuit with 18 operations at 8 points and none with 51 at 16, which proves nothing. I found no exact minimum for 8 or 16 points in print.

The count of the 1994 paper

Remark 7 (every line is one multiply/add). One operation of [4] is $\pm a \pm bc$, with any of the four choices of the two signs, or a plain addition or multiplication. Every line of the count-(G) circuits of this paper is $\pm u \pm v$ or $\pm u \pm 2v$: it has two terms and at most one multiplication by a constant. This can be read off Table 2, the procedure $E(X, s)$ and the proof of Theorem B, and `check.py` tests it for every line of the 19 and of the family that it makes. Some lines have a minus sign on the term that is not doubled: they are $-u+2v$ or $2u-v$. There are 2 such lines among the 19 (the lines y_0 and y_5) and 12 among the 52 of the family at 16 points. Such a line is the form $-a+bc$, so it is one operation of [4] as well. Hence 19, 52 and $\frac{3}{4}N\log_2 N + \frac{1}{4}N$ are counts in the sense of [4] as they stand. Count (G) as the Introduction defines it is wider in two ways: it allows a constant on both operands, and it lets a multiplication on its own go free, where [4] charges one. The count-(G) circuits of this paper use neither.

Two remarks without proofs

Remark 8 (other 2×2 matrices). H_N is the k-fold tensor power of the matrix with rows $(1,1)$ and $(1,-1)$: it is made from that matrix by $k-1$ steps of the block rule for H_{2N} in the Introduction. Put another 2×2 matrix with four non-zero entries in its place; after scaling its rows and columns it has rows $(1,1)$ and $(1,\tau)$. Programs of the agents report that, with τ kept as a symbol, the wiring of the family of Section 4 at 4, 8 and 16 points, and the wiring of the 19-operation circuit after a renaming of the inputs and outputs and a scaling of each output, compute the corresponding transform with other constants. This is computed, not proved, and no person has checked these programs. If it holds, the saving is not special to the Walsh–Hadamard transform.

Remark 9 (a restricted class). Every value of the circuits in this paper is, up to one factor, a vector with entries $0, \pm 1$. Call the number of an input, written with k binary digits, its *address*. In my working notes there is an argument that if in addition the support of every value is a node of one fixed tree that cuts the set of all addresses into two halves by the value of one digit, and each half again, down to single addresses, then $N-1+\frac{1}{2}N\log_2 N$ operations are needed: 7, 19 and 47 at 4, 8 and 16 points. The 19-operation circuit is of that kind. No person has checked this argument, so I state it as a direction and not as a result.

The circuits as hardware

Remark 10 (count (G) as a count of adders; measured, not proved). In hardware built from *adders* (an adder adds or subtracts two binary numbers) doubling is a shift by one place, which is wiring. So each line of the count-(G) circuits of this paper is one adder, and their count is the number of adders: 19 against the textbook's 24 at 8 points. The agents of Section 7 built the circuits and the textbook network for *FPGAs*, chips whose logic is set by loading a file into them and which are made of units called *logic cells*. They used two open-source tools: Yosys 0.69 for synthesis, which turns a design into logic cells, and nextpnr 0.11.1 for placement and routing, which fits the cells onto the chip and wires them. Placed and routed for a Lattice iCE40HX8K, as one block of adders with no *pipeline registers* inside (rows of one-bit memories between the levels of adders, which let the clock run faster), with 8-bit inputs and one output register, the 19-adder circuit takes 274 logic cells where the textbook network takes 349, which is 75 fewer (21.5 %), or 273 against 337 (64 fewer, 19.0 %) with the synthesis option `-dff`. The family of Theorem B, built as such a block with signed 9-bit inputs and no output register, takes 17.5 % fewer cells than the textbook network at 16 points, rising to 23.1 % fewer at 256, on the iCE40 (counted before placement; from 128 points on the designs do not fit that chip), and 19.1 to 22.0 % fewer on a Lattice

ECP5. With an output register and 8-bit inputs the 16-point circuit takes 877 cells against 969 (9.5 % fewer), and the 56-adder design of [4] takes 720.

The saving has a price. The circuits are twice as deep (6 levels of adders against 3 at 8 points), so the tools estimate the highest clock rate of the 8-point block, the number of results it can take a second, at about 72 MHz against about 97 MHz for the textbook (timing estimates, not measurements), and the family's blocks as about 30 % slower than the textbook's on the iCE40 and about 37 % slower on the ECP5. Once pipeline registers are added for speed, the textbook is the smaller design: above about 85 MHz at 8 points (with a register after every adder, 677 cells against 363), and with a register after every level of adders the family needs 67 to 70 % more iCE40 cells than the textbook. The 19-adder circuit is also not the smallest 8-point design on this chip: the 22-adder step of Alman and Rao [3] takes 244 cells under the same conditions. With these tools a subtraction costs more than an addition on this chip, and about half of the saving over the textbook comes from having 9 subtractions against 12; their step has one. Without the output register the family takes 10 to 14 % fewer cells than the design of [4] on the ECP5 and about as many on the iCE40 (between 2 % fewer and 10 % more, depending on the size). On an iCE40-HX8K FPGA board running at 12 MHz, I ran my 19-adder 8-point Walsh-Hadamard circuit side by side with the textbook circuit on 597,750,000 test inputs, and the two gave the same outputs every time; a deliberately broken copy of the circuit, run alongside, was caught, which shows the test can fail. This checks only that the circuit computes the right values on a real chip: it says nothing about size or speed, and in this pipelined build the 19-adder circuit is the larger of the two designs. The lines that the board sent are in the repository, in `tools/circuits/fpga/board/run-2026-10-11/`. These are measurements and estimates by the agents with one open-source tool chain; the chip makers' own tools were not run, and nothing was measured for custom silicon or for power. The files and the conditions are in the folder `tools/circuits/fpga/` of the repository [17]; those of the timings of Section 8 are in `tools/circuits/fast/wht56/results/ffht/`.

7 How the circuits were found, and the use of AI

The circuits were found on the night of 10 to 11 October 2026 by computer searches that were written and run by AI coding agents (Anthropic's Claude) under my direction. They were a by-product of a search for something else, a transform of linear size, which found nothing of that kind. The working reports of that search, about 180 of them with their programs, are in the folder `waves/` of the repository [17]; its page `WHERE-THE-CIRCUITS-CAME-FROM.md` names the report behind each step described here, and `LEADS.md` lists what the search left open. One step of that search was to try to prove that $N \log_2 N$ operations are needed when a circuit holds only pieces in the sense of Section 4. A local search in a slightly larger set of values, started from a 22-operation circuit, fell to 20 operations at 8 points. The 20 contained a value that is not a piece: the hub of Lemma 4. Used as a rule at every size, it is the family of Theorem B. A random search over sets of helper values (simulated annealing), again started from the 22, then fell to 19 in each of four runs and never to 18. The Lean proof of Theorem A, the programs, the literature search, the proofs and this text were also written by the agents. I am responsible for the content of this paper. The agents also ran the programs and the checks of the repository, and made the measurements of Section 8 and Remark 10, at my direction, on a Mac mini of mine and on a Linux machine that I rented. No person has yet read the Lean files or checked the proofs line by line.

The literature search was also made by the agents: the papers that the citation indexes list as citing [3] or [4], most of them seen by title and abstract only; the text of [3], [5], [19] and [8]; several dozen queries to arXiv and to web search engines; and seven patents, read through a tool that summarises pages. The two older papers [4] and [7] were read by the agents, from copies I obtained. The first has two pages, and both were read. Of the second the agents read page 172 (the introduction) and pages 178 to 183: the comparison tables, the paragraph on the count for the Hadamard transform, the conclusions, and the appendix with the proof of the count and the programs. Pages 173 to 177 were not read. By the outline on page 172 they hold the part on the Fourier transform and also the section that sets out the algorithm for the Hadamard transform, so the algorithm is taken here from the proof and the program in the appendix. Nothing found has 8 points below 22 operations, 16 points below 56, a rate below 7/8 in count (G), or a rate below 23/24 in count (O); the count stated in [7] is not such a result, for the reason given in the Introduction.

What limits the word "new". The paper [13] is known from its abstract only. The same holds for three works of Sarukhanyan, Agaian, Astola and Egiazarian on Hadamard transforms and multiply-add architectures [14, 15, 1]; their abstract speaks of beating the classical transform for orders 96 and above, and what is compared could not be told from it. A survey by I. S. Sergeev [16] has a section called

"Linear circuits for Sylvester matrices" (H_N is also called the Sylvester matrix); only the first 49 pages of the survey were read, and that section is not in them. The comment of Frigo and Johnson [6] is known from its abstract only, and the paper of Morgenstern [9] from later accounts. Five pages of [7] were not read, as said above. No search was made for lower bounds like Proposition 6. Literature in languages other than English was seen only as far as the indexes returned it. If you know a smaller count in print, I would be glad to hear of it and will credit it.

8 What this is not

- **It is not a better exponent.** Every count here is a constant times $N \log_2 N$. The results of [10, 11, 18] are of smaller order. The count-(G) circuits of this paper cannot reach a smaller order as they are: their constants are ± 1 and ± 2, and for circuits whose constants are bounded, Morgenstern's determinant argument [9] gives a lower bound of order $N \log N$.
- **It is not a statement about running time.** No result of this paper is a timing, and where time was measured, fewer operations did not mean less time. The test that counts is against FFHT, a well-known fast open-source library for this transform, whose code is written out as *vector instructions* (processor instructions that work on several numbers at once). The 8-point step was written into the code that FFHT's own generator produces, in place of its block of 24 additions and subtractions, and nothing else was changed. On the one machine that was timed, this did not make FFHT faster at any size from 2^6 to 2^{26} points in single precision or from 2^5 to 2^{26} points in double precision: set against a second build of FFHT's unchanged code, all 129 combinations of size and form took more time.[1] One of the three forms that were tried has the 19 lines of count (G) as 19 instructions: each line with a doubling is done by a *fused multiply-add*, a single instruction that computes $a + b \times c$. It was not faster either. The likely reason is the order in which the operations depend on each other: in the textbook block no result is more than 3 operations away from the loaded numbers, and in the new step the last results are 7 operations away, 6 in the fused form. What happens inside the processor was not measured, so this is an explanation that fits, not a finding. In my own plain C, where the compiler writes the vector instructions, the new step took less time than the textbook step on that machine only in double precision at the sizes $2^6, 2^9, \ldots, 2^{27}$; it took more time at every other size from 2^4 to 2^{26}, and in single precision at every size. At 2^6 to 2^{18} points that gain is larger than replacing 24 operations by 20 can give, so it comes from how the compiler treats one pass of my code and not from the operation count; the cause has not been found. Other timings of plain C, on that machine and on a Mac mini, are beside the programs in the repository [17]; in no build did the time follow the operation count. Only the 8-point step was timed, not the larger circuits of the family. The 1994 paper shows the same gap between counting and timing: its count predicts a gain of 12.5 % over an ordinary routine of the same kind, and the authors measured 5.6 % to 7.4 % on their machine, an IBM RISC System/6000 Model 530, at 64 to 4,096 points [4, pages 969 and 970].
- **It is not known to be optimal.** The only lower bound proved here is the $2N-1$ of Proposition 6, one more than the obvious $2N-2$. In count (O) I do not know whether the exact transform on fewer than 512 points can be computed with fewer than $N \log_2 N$ operations. Nor do I know whether the rate 5/6 can be lowered. An attempt to bring the rate 3/4 of the family into count (O) found nothing below 5/6 (Remark 3: computations and an argument of the agents, not in Lean). A direct search for a 16-point circuit with 53 or fewer operations of count (O) has not been made.
- **The method is not mine.** Splitting a small transform into a part of low rank and a sparse part, and the single halving, are how Alman and Rao obtain their 8-point step. The 4-point circuit with

[1] Conditions. One rented virtual machine with the AVX2 and FMA sets of vector instructions, which reports itself as an Intel Xeon at 2.20 GHz, with other programs running on it at full load the whole time; the compiler gcc 13.3.0, for the plain C with `-O3 -march=native`; FFHT, https://github.com/FALCONN-LIB/FFHT, at its last version (March 2019), built by its own Makefile, with the plan of passes it ships; one thread, kept on one processor; wall-clock time, the best of 15 repeats of a loop of 0.2 s or more (9 repeats where a size was timed a second time, and for the plain C); 11 October 2026. The three forms: the 20 operations on the right of Table 2 with the output left divided by a power of two; the same with multiplications added so that the output is exact; the 19 lines on the left with fused multiply-adds, exact. From 2^7 points (2^6 in double precision) to 2^{19} points the changed FFHT took between 1 and 24 % more time, depending on the form and the precision, and from 2^{20} points 2 to 7 % more on average, which for the fused form is too small a difference for one run to establish. Plain C: 7 to 37 % less time with exact output at the sizes named, 1 to 13 % more at every other size from 2^4 to 2^{26} in double precision, and 2 to 51 % more at every size from 2^3 to 2^{27} in single precision; 20 operations in place of 24 can save 17 % at most. The timings were made by the agents of Section 7, in one run unless said otherwise, and have not been repeated on a second machine, with a second compiler, or by anyone else.

7 operations is due to Coppersmith, Feig and Linzer [4]. What the searches added is which values to share.

References

[1] S. Agaian, H. Sarukhanyan, K. Egiazarian and J. Astola, *Hadamard Transforms*, SPIE Press, Bellingham, 2011. (Abstract only.)

[2] J. Alman, Kronecker products, low-depth circuits, and matrix rigidity, in *Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing (STOC 2021)*, pages 772–785. doi:10.1145/3406325.3451008; arXiv:2102.11992.

[3] J. Alman and K. Rao, Faster Walsh-Hadamard and discrete Fourier transforms from matrix non-rigidity, in *Proceedings of the 55th Annual ACM Symposium on Theory of Computing (STOC 2023)*, pages 455–462. doi:10.1145/3564246.3585188; arXiv:2211.06459, version 2.

[4] D. Coppersmith, E. Feig and E. Linzer, Hadamard transforms on multiply/add architectures, *IEEE Transactions on Signal Processing* 42(4) (1994), 969–970. doi:10.1109/78.285664.

[5] E. Feig, Complexity theory of transforms in signal processing, Chapter 9 of *The Digital Signal Processing Handbook*, edited by V. K. Madisetti and D. B. Williams, CRC Press, Boca Raton, 1999.

[6] M. Frigo and S. G. Johnson, Comments on "Split manageable efficient algorithm for Fourier and Hadamard transforms", *IEEE Transactions on Signal Processing* 55(4) (2007), 1547–1548. doi:10.1109/TSP.2006.889087. (Abstract only.)

[7] A. M. Grigoryan and S. S. Agaian, Split manageable efficient algorithm for Fourier and Hadamard transforms, *IEEE Transactions on Signal Processing* 48(1) (2000), 172–183. doi:10.1109/78.815487. (Pages 172 and 178 to 183 only.)

[8] A. M. Grigoryan and V. S. Bhamidipati, Method of flow graph simplification for the 16-point discrete Fourier transform, *IEEE Transactions on Signal Processing* 53(1) (2005), 384–389. doi:10.1109/TSP.2004.838970.

[9] J. Morgenstern, Note on a lower bound on the linear complexity of the fast Fourier transform, *Journal of the ACM* 20(2) (1973), 305–306. doi:10.1145/321752.321761. (Not read; known through later accounts.)

[10] OpenAI, Finite tensor savings and exact Fourier circuits, preprint of 25 September 2026, with a Lean development, in the folder `preprints/` of the repository `https://github.com/openai/math`.

[11] OpenAI, An explicit power saving for the exact discrete Fourier transform, preprint of 25 September 2026, in the folder `preprints/` of the repository `https://github.com/openai/math`.

[12] E. Reshef and I. Alrod, Reduced complexity fast Hadamard transform, United States patent 7,003,536 B2, filed 15 August 2002, granted 21 February 2006. (Read through a tool that summarises pages.)

[13] S. Samadi, Y. Suzukake and H. Iwakura, On automatic derivation of fast Hadamard transform using genetic programming, in *Proceedings of the 1998 IEEE Asia-Pacific Conference on Circuits and Systems (APCCAS 1998)*, pages 327–330. doi:10.1109/APCCAS.1998.743763. (Abstract only.)

[14] H. Sarukhanyan, S. S. Agaian, J. T. Astola and K. O. Egiazarian, Binary matrices, decomposition and multiply-add architectures, in *Image Processing: Algorithms and Systems II*, Proceedings of SPIE 5014 (2003), 111. doi:10.1117/12.473134. (Abstract only.)

[15] H. Sarukhanyan, S. Agaian, J. Astola and K. Egiazarian, Decomposition of binary matrices and fast Hadamard transforms, *Circuits, Systems and Signal Processing* 24(4) (2005), 385–400. doi:10.1007/s00034-004-0811-y. (Abstract only.)

[16] I. S. Sergeev, Etudes on the methods of designing efficient circuits, a survey of about 170 pages, version of 28 May 2025, on ResearchGate. (Pages 1 to 49 only.)

[17] J. Sussman, *wht-fewer-operations*, the repository of this paper, with the circuits, the programs, the Lean files and the source of the paper, 2026. `https://github.com/jacobalansussman/wht-fewer-operations`.

[18] J. Sussman, *wht-power-saving-lean*, an earlier repository of Lean proofs and programs, 2026. `https://github.com/jacobalansussman/wht-power-saving-lean`.

[19] Y. Voronenko and M. Püschel, Mechanical derivation of fused multiply–add algorithms for linear transforms, *IEEE Transactions on Signal Processing* 55(9) (2007), 4458–4473. doi:10.1109/TSP.2007.896116.